

Map in Building the PINN

Problem ~~Question~~ Definition:

$$\frac{\partial y}{\partial t} = \frac{\partial^2 y}{\partial x^2} - e^{-t} [\sin(\pi x) - \pi^2 \sin(\pi x)] \quad (1)$$

$$x \in [-1, 1]$$

$$t \in [0, 1]$$

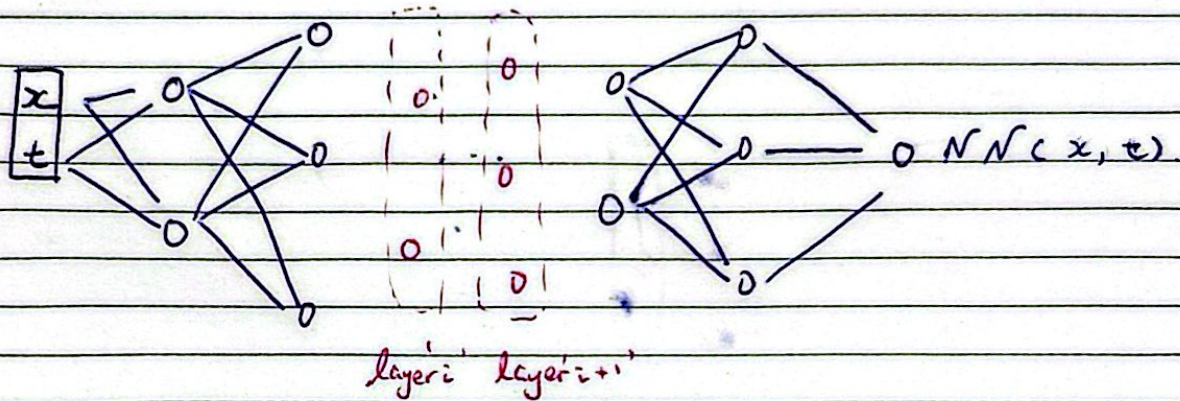
$$\text{IC: } y(x, 0) = \sin(\pi x)$$

$$\text{BC: } y(-1, t) = 0$$

$$y(1, t) = 0$$

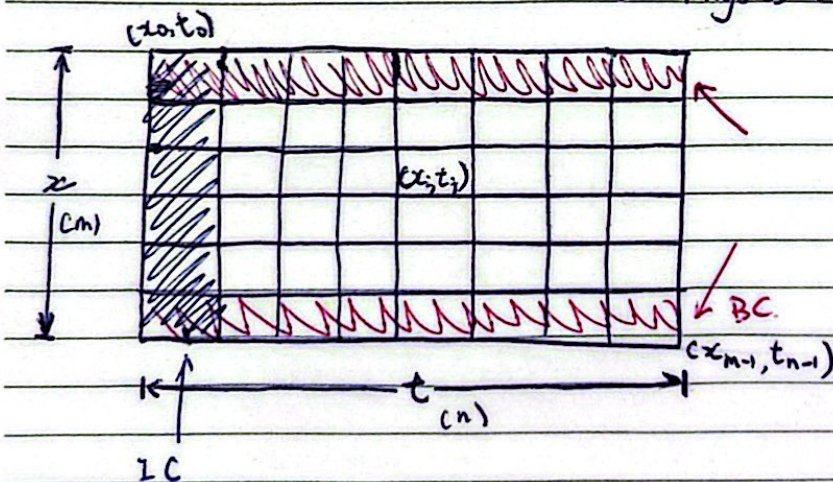
$$\text{Exact Solution: } y(x, t) = e^{-t} \sin(\pi x)$$

Neural Networks:



PINN training using 1) NN $\rightarrow$ data from ICs and BCs.

2) Physics (PDE) $\rightarrow$ data from field & IC & BC



$$X = \text{torch.linspace}(-1, 1, m)$$

$$T = \text{torch.linspace}(0, 1, n)$$

$$X, T = \text{torch.meshgrid}(X, T)$$

$$y_{\text{exact}} = y(X, T)$$

1) NN Training using data from ICs & BCs: (2 sets of data)

x_n, t_n (tensors from ICs & BCs)

y_n (y given by ICs & BCs)

$NN(x_n, t_n)$ (NN given by FP)

Penalising deviations from given ICs & BCs

$$\text{MSE}_{NN} = \frac{1}{2} \sum_{i=0}^{n-1} [y(x_i, t_i) - NN(x_i, t_i)]^2$$

2> Physics Training data from the whole field: $C\alpha + \beta$ secs of data)
 $\underline{x}_p, \underline{y}_p$ (tensors randomly selected from the field).

$NN(\underline{x}_p, \underline{y}_p)$ CNN given by ~~forward~~ FP) (also add training points from ICs & BCs).

Penalise deviation from the physics (ODE):

$$\text{modify } \llbracket j = \frac{\partial y}{\partial t} - \frac{\partial^2 y}{\partial x^2} + e^{-t} [\sin(\pi x) - \pi^2 \sin(\pi x)] = 0. \llbracket 2.$$

$$\text{Similarly: } f = \frac{\partial NN}{\partial t} - \frac{\partial^2 NN}{\partial x^2} + e^{-t} [\sin(\pi NN) - \pi^2 \sin(\pi NN)] \approx 0 \llbracket 3$$

$$MSE_P = \frac{1}{\alpha + \beta} \sum_{i=0}^{\alpha + \beta - 1} [f - 0]^2.$$

In general, $MSE = MSE_{NN} + MSE_P$

Define the NN for the problem:
 class FCN(Cnn.module):

① `def __init__(self, layers):`

a> initialise super class

b> define activation func.

c> define loss func. using nn.MSELoss (reduction = 'mean')

~~d> define (some) layers~~

d> define linear layers. (how many neurons on each layer).

e> initialise value on each neuron

$l_0, l_1, \dots, l_{p-1}, l_p$

② `def forward_propagation(self, x):`

$$NN(x) : x \mapsto NN(x).$$

③ `def loss_func`

`def loss_BC`

`def loss_P`

define loss function as a summation of both BC & P.

OPTIMISATION: